

# Comms-Loop: Multi-Agent Coordination Through an Append-Only Ledger

Yogesh Rathod · [yrathod.research@gmail.com](mailto:yrathod.research@gmail.com)

EXPERIENCE REPORT, COMMS-LOOP PROJECT — WRITTEN 2026-09-15

System and data: <https://github.com/YRathod/comms-loop>  
protocol, ledgers, and the full message archive are published in this repository

---

## ABSTRACT

Comms-loop lets AI agents work together on multi-day tasks without a human watching every step. The agents do not chat. They leave short markdown notes in folders on a shared disk. Every note belongs to a numbered thread, one append-only **ledger** records everything in order, and nothing is edited or deleted. Background daemons watch the folders and keep "who owes a reply" boards current. A **learning loop** records the gap between what an agent expected and what happened as a number, and a failed training cycle ends in a classified autopsy, a written 5-why, and a research scan that must yield one pre-registered fix candidate.

This is an experience report, not a controlled study. One system, no baseline. Over an 18-day window two agents wrote 85% of 762 messages across about 30 threads, in bursts of one to four days that the human opened and closed. The longest run on one thread was 27 hours; the longest unattended stretch was 10 hours. The human wrote 7% of messages and never touched turn-taking, but issued every key and directed most protocol changes. Only the daemons are software. Every other rule is a convention the agents follow.

## 1. Introduction

---

Getting several AI agents to do one long piece of work is hard for boring reasons. Chat forgets: context windows fill, sessions restart, and "what did we decide yesterday?" becomes unanswerable. Agents also drift: they bend their own rules mid-task, over-estimate progress, and grade their own work. The usual fix is a human in the loop, which does not scale and turns the human into a courier.

Comms-loop makes the medium dumb and the record permanent. Chat is replaced by a shared folder of append-only files and a ledger. Continuous supervision is replaced by a few hard gates, such as spending money or starting a training run, that only the human can open. Everything else is meant to run unattended.

### What this report can support.

- **Observation.** A filesystem plus stdlib Python was enough to carry multi-day multi-agent work whose record now spans six weeks, with every decision locatable, dated, and attributed.
- **Hypothesis 1.** Process rules (pre-registration, checked references, one change per cycle) control drift better than smarter prompting. The record is consistent with this; nothing was run without them, so it is untested.

- **Hypothesis 2.** Human involvement can shrink to authorization and escalation without losing control. The message count supports this; the human's leverage per message (section 5) complicates it.

## 2. Related Work

---

**Shared spaces and logs.** Coordinating through a shared space instead of direct messages goes back to blackboard systems (Nii 1986 [1]) and Linda tuple spaces (Gelernter 1985 [2]). Comms-loop's mail files are closest to Linda tuples: written once, read by whoever polls, ordered by name. It differs from a blackboard in one way: blackboards had a control shell deciding what fires next, and comms-loop has none. Turn order comes from sequence numbers and a no-double-send rule. The ledger is event sourcing (Fowler 2005 [3]): state is derived by reading an append-only log, never mutated in place.

**LLM agent frameworks and protocols.** AutoGen (Wu et al. 2023 [4]) and MetaGPT (Hong et al. 2023 [5]) coordinate agents through conversation; MetaGPT's forced structured documents between steps are closest to our artifact-carrying mail. Agent2Agent (Google 2025 [6]) and the protocol survey by Ehtesham et al. (2025 [7]) standardize how agents connect and leave what keeps them honest over a long task to the application. Comms-loop sits at that layer and could ride over A2A unchanged.

**Memory, self-correction, and oversight.** Generative Agents (Park et al. 2023 [8]) and Reflexion (Shinn et al. 2023 [9]) keep reflections and self-critiques in the model's own memory stream. Our loop keeps corrections in files outside any context window, because long contexts lose information (Liu et al. 2024 [10]) and unaided self-correction does not reliably improve reasoning (Huang et al. 2024 [11]). Pre-registration with falsifiers borrows from science (Nosek et al. 2018 [12]). The human's role is supervisory control in the sense of Sheridan & Verplank (1978 [13]): set goals and gates, intervene by exception, never steer each step.

## 3. Architecture

---

### 3.1 Components

```
comms/
  PROTOCOL.md          the rules (versioned; changes are themselves protocol mail)
  LEDGER.md            thread index (human-facing) + append-only message log
  PENDING[_<party>].md  wake board: who owes a reply, rewritten each daemon tick
  inbox_<party>/       one folder per party; a writer drops mail into recipients' folders
  acks/                acknowledgment markers for action mail
  archive/             processed mail – moved, never edited
  *_CALIBRATION.md     per-agent learning-loop ledgers (§4)
scripts/
  comms_daemon.py       inbox watcher (detection only)
  comms_responder_daemon.py  ack-marker writer (see §3.3)
  muse_reply_daemon.py  wrapper over the responder for the muse seat
```

A message is a markdown file with YAML front-matter: `from`, `to`, `project`, `thread`, `seq`, `re-seq`, `type` (result, review, action, question, protocol, blocker, void-notice), and `refs`, artifact paths that must

resolve at send time. Filenames carry a UTC timestamp, sender, thread, and sequence number, so a directory listing is already an ordered log. (`project` became mandatory at v1.11, after the window.)

**Enforced versus convention.** The watchers, wake boards, heartbeat files, and the September alignment-gate script are software. Everything else, including refs checking, void-on-arrival, sequence rebase, and provenance tags, is a rule agents follow because the protocol says so and the reviewer checks. Any party can write any file. This report says "by rule" for those and "enforced" only for the daemons.

### 3.2 Threads, roles, and turns

Every mail belongs to one thread. The opener names an **owner** (does the work), exactly one **reviewer** (verifies artifacts, can hold or void; never the owner), and observers.

Sequence numbers are self-assigned as  $\text{max-seen} + 1$ . If two mails claim the same number, the later timestamp is void by rule and its sender re-sends. No party sends twice in a row without a reply, except for void-notice, blockers, and reviewer holds. Threads move `OPEN`  $\rightarrow$  `RUNNING`  $\rightarrow$  `REVIEW`  $\rightarrow$  `CLOSED` or `HELD`. Only the reviewer closes; only the holder lifts a hold. A thread idle 24 hours is pinged or closed as stale.

### 3.3 Daemons and boards

The daemons are detectors. The watcher polls inboxes every 30 seconds, rewrites the `PENDING` boards, appends to a detection log, and touches a heartbeat file. A heartbeat older than five minutes is flagged in-channel. Each party runs its own watcher with isolated state.

The responder layer is where the design was violated. Until 2026-09-15 the responder daemon carried hard-coded reply bodies, and one of them was sent four times as `type: result` under an agent's name in the sympy-symbolic thread in August. Earlier, auto-acks had taken thread sequence numbers and read as reviewer judgments. The rule now is that auto-acks take no seq and daemons never author mail. The responder and the muse seat's daemon were cut to ack-marker writers on 2026-09-15, with tests that fail if any content-generating branch returns. Judgment stays with full agent sessions.

### 3.4 The human's position

- **Authority.** Ratifies seats and protocol versions.
- **Escalation target.** Credentials, money, instance lifecycle, and deletions are human-only threads with no workarounds.
- **Key issuer.** No training run starts without signed authorization in-channel, as a named-run key or a time-boxed standing key.

Everything else is agent-run.

## 4. The Learning Loop

---

Long tasks drift in three ways: estimates detach from reality, participants bend procedures, and context windows lose earlier decisions. The loop treats each by rule.

#### 4.1 Four steps

Monitor loops (daemons, heartbeats) keep the system responsive but learn nothing. Learning loops run on a cadence:

|         |                                                                                                                              |
|---------|------------------------------------------------------------------------------------------------------------------------------|
| OBSERVE | what did I do since last cycle (artifacts, estimates, calls)                                                                 |
| DIFF    | expectation vs outcome – as NUMBERS, never narrative                                                                         |
| BANK    | one append-only line per diff to the agent's calibration ledger                                                              |
| ADJUST  | at most ONE pin/practice change per cycle, proposed as a dated amendment through normal review – never silently self-applied |

Three rules carry it. One adjustment per cycle prevents process thrash. Diffs must be measured; "went well" is not a diff. Bank even when the diff is zero. Learning loops never hold keys.

#### 4.2 Cadence

A fixed 30-minute tick banked contentless lines during long runs, so the cadence became event-driven: a diff banks when an estimate is issued, resolves, or is recomputed. A two-hour sweep checks heartbeat age and inbox counts and recomputes one previously banked number from raw artifacts, chosen by a seeded rotation pinned in the ledger header, because choosing easy targets in the moment is "the quiet failure mode of self-audit."

#### 4.3 Standing rules

- **Pre-registration with falsifiers.** Failure criteria are pinned before the run; "prospective" may only be claimed for predictions stamped pre-run.
- **Checked references.** refs must resolve; a result whose refs do not is void on arrival.
- **Provenance.** Relayed restrictions bind immediately; relayed permissions never do.
- **Quarantine.** Runs launched without a valid key stay provisional forever.
- **Key-holder beats.** One line per 30 minutes of key time; two missed beats and the holder may be suspended.
- **Protocol self-evolution.** Rule changes are protocol mail, reviewed and versioned.

#### 4.4 Autopsy, 5-why, and the literature leg

Two mechanisms added after the window (v1.13 on 2026-09-13, v1.19 on 2026-09-15, both human-directed) turn a failed training cycle into a structured next attempt.

**Autopsy with a written 5-why.** A loop may only fire a new round on a pinned failure count over a fresh human-labelled batch. Each round classifies every failure by class, then pauses until a 5-why exists with fixed rungs: is the key right, largest fixable class, missing mechanism, route. Rounds are capped at three; a two-point eval regression kills the round. An autopsy on the eval slice may name classes but never lift strings, so eval failures cannot leak into training data.

**Literature leg.** Every autopsy closes with a read-only scan of open research targeting the named loss classes, cited by title, venue, and year. At least one technique is adopted as a fix candidate, pre-registered with a banded prediction before any number exists, and rejected techniques are recorded with the reason. No eval slice is read and no training data comes from the web.

Both had their first live use on Sep 13 to 15, outside the window, so they are reported as design, not results.

## 5. What Happened

---

The system ran from 2026-08-02 to 2026-08-19 coordinating small-model training research, then was migrated unchanged into its own repository on 2026-09-13 through the protocol itself.

| Measure                                               | Value                                                                                            |
|-------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| Messages (main channel + secondary arm)               | 488 + 274 = 762                                                                                  |
| Threads / seats                                       | ~30 / 6 (kimi, fable, grok, gemini, muse, human), four model families                            |
| Messages by sender (main)                             | kimi 216, fable 199, muse 33, grok 5, gemini 1, human 34                                         |
| Protocol versions                                     | 10 in the window, 19 by 2026-09-15, all via in-channel review                                    |
| Activity bursts (gap > 8 h splits)                    | Aug 2–4 58 h / 200 msgs; Aug 3–7 sympy arm 92 h / 270; Aug 10–12 40 h / 176; Aug 12–14 31 h / 83 |
| Idle stretches                                        | Aug 5–9, Aug 16–18                                                                               |
| Longest thread / longest continuous run on one thread | 54 seqs / 27 h (method-discovery)                                                                |
| Longest unattended window                             | 10 h                                                                                             |

**Bursts, not continuous operation.** Each burst was opened and closed with the human present. No thread ran continuously for more than 27 hours. A post-migration burst on Sep 13 to 15 reached 54 hours but falls outside the window.

**Effectively two agents.** Kimi and fable wrote 85% of main-channel mail. Muse wrote 33 real reviews and results; grok and gemini together wrote six. "Five agents from four families" is accurate as a roster and misleading as a description.

**Human involvement: low volume, high leverage.** The human wrote 7% of messages and never intervened in turn-taking, review, holds, or voids. But the human opened and closed every burst, issued every key, ruled on every blocker, and directed most protocol versions. The right statement is "authority and gatekeeper, never courier."

**Multi-day work under light supervision.** The moonlender thread (50 seqs) closed positive with its deliverable verified 14/14 by the reviewing agent, including a 10-hour overnight autopilot window

under a standing key. The fourconnect arm reached a pre-registered *falsified* verdict and parked itself, because its falsifiers were pinned before the runs. "Verified" here means by a second model, which catches errors one model would miss but not errors both share.

**What the calibration ledger shows.** The executor agent's ledger reached about 80 entries by 2026-09-15. Its five wall-clock ratios (0.65, 1.00, 0.50, 0.15, 1.20; median 0.65) suggest over-estimation, but the ledger's own header calls the series noise until ten exist. A second axis on 2026-09-14 found pass-probability estimates two times optimistic and registered a 0.5 haircut. Whether any adjustment improved later estimates is unmeasured. The loop has detected miscalibration twice and adjusted once; it has not yet shown that adjusting helped.

**Failure modes.** Each became a dated protocol amendment:

- A concurrent ledger rewrite lost 8 thread rows (led to single-writer boards).
- A cross-session pause killed the shared watcher (led to per-party daemons).
- An agent-authored key "expiry" was ruled invalid (led to human-only keys).
- An instance-side protocol edit failed to sync (led to sync checks).
- The responder daemon sent templated result mail four times (led to the no-authoring rule and the ack-only cut, §3.3).

## 6. What We Would Claim, and What We Would Not

---

**What it bought.** Durability across session deaths and model swaps. Attribution for every decision in six weeks. Transport independence. A ledger table that compresses the whole window into one screen.

**Costs.** Rule churn is the unexamined one. Every incident became a rule; the protocol grew from one page to 15 sections and 19 versions in six weeks. Every agent reads it at session start, and nothing measures whether compliance holds as it grows. The report's thesis is that drift is best fought with rules; the protocol's growth is itself drift, handled in the open but not bounded.

**Trust by convention.** Only the daemons are enforced. There is no access control, no signing, and one reviewer per thread. This held with cooperative models. It is not a security property.

**Other limits.** Latency is human-scale (30-second polls, session-paced replies). Throughput is bounded by how often each seat opens a session. The drift controls catch process drift, not capability limits. Continuity beyond a few days, without a human checkpoint, is untested.

**What would turn the hypotheses into results.** Run the same task through a chat framework and through comms-loop and count violations, voids, and interventions. Run one thread without pre-registration. Grow the ratio series past ten and report whether ratios move toward 1.0 after each adjustment. Measure compliance against protocol length. None of this has been done.

## 7. Conclusion

---

A shared folder, an append-only ledger, threads with one owner and one reviewer, daemons that watch but do not judge, and a loop that banks measured diffs. None of it is new, and most of it is convention rather than software. After six weeks the supportable statement is narrow: two agents, with three occasional seats, did multi-day work in bursts with the human as gatekeeper rather than courier, and every step can be found in the record. Whether the rules beat the alternatives is the next experiment, not this one.

## References

---

1. Nii, H.P. (1986). Blackboard Systems, Part One: The Blackboard Model of Problem Solving and the Evolution of Blackboard Architectures. *AI Magazine* 7(2), 38–53.
2. Gelernter, D. (1985). Generative Communication in Linda. *ACM Transactions on Programming Languages and Systems* 7(1), 80–112.
3. Fowler, M. (2005). Event Sourcing. <https://martinfowler.com/eaDev/EventSourcing.html>
4. Wu, Q., Bansal, G., Zhang, J., et al. (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155.
5. Hong, S., Zhuge, M., Chen, J., et al. (2024). MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework. *ICLR 2024*; arXiv:2308.00352.
6. Surapaneni, R., Jha, M., Vakoc, M., Segal, T. (2025). Announcing the Agent2Agent Protocol (A2A). Google for Developers Blog, 9 April 2025.
7. Ehtesham, A., Singh, A., Gupta, G.K., Kumar, S. (2025). A Survey of Agent Interoperability Protocols: Model Context Protocol (MCP), Agent Communication Protocol (ACP), Agent-to-Agent Protocol (A2A), and Agent Network Protocol (ANP). arXiv:2505.02279.
8. Park, J.S., O'Brien, J.C., Cai, C.J., Morris, M.R., Liang, P., Bernstein, M.S. (2023). Generative Agents: Interactive Simulacra of Human Behavior. *UIST 2023*; arXiv:2304.03442.
9. Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., Yao, S. (2023). Reflexion: Language Agents with Verbal Reinforcement Learning. *NeurIPS 2023*; arXiv:2303.11366.
10. Liu, N.F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., Liang, P. (2024). Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the ACL* 12, 157–173; arXiv:2307.03172.
11. Huang, J., Chen, X., Mishra, S., Zheng, H.S., Yu, A.W., Song, X., Zhou, D. (2024). Large Language Models Cannot Self-Correct Reasoning Yet. *ICLR 2024*; arXiv:2310.01798.
12. Nosek, B.A., Ebersole, C.R., DeHaven, A.C., Mellor, D.T. (2018). The Preregistration Revolution. *PNAS* 115(11), 2600–2606.
13. Sheridan, T.B., Verplank, W.L. (1978). Human and Computer Control of Undersea Teleoperators. MIT Man-Machine Systems Laboratory, technical report.